Générateurs de Nombres Pseudo-Aléatoires

Implémentation, Tests et Attaques

Génération de Nombres Aléatoires

MAHE Eliot GALLIC Maxime OZDEMIR Denis

18 février 2026

Informations administratives

Élément	**Détail**
Auteurs	MAHE Eliot, GALLIC Maxime, OZDEMIR Denis
Date	18 février 2026
Établissement	ENSIBS
Promotion	ICE 3A promotion 2028
UE	Génération de Nombres Aléatoires

Résumé

Ce rapport présente l'étude comparative de sept générateurs de nombres pseudo-aléatoires (PRNG), allant des algorithmes classiques non cryptographiques aux générateurs certifiés par le NIST. Nous implémentons et analysons le LCG, le Mersenne Twister, la transformée de Box-Muller, le générateur Blum-Blum-Shub, le Hash_DRBG (NIST SP 800-90A), l'interface système `os.urandom` et un générateur hybride XOR NRBG. Quatre tests statistiques sont mis en œuvre pour évaluer la qualité de leurs sorties : l'entropie de Shannon, le test du chi-carré, l'autocorrélation et le test de Kolmogorov-Smirnov. Enfin, deux attaques pédagogiques démontrent la vulnérabilité des générateurs non cryptographiques : la récupération algébrique de la graine d'un LCG et la reconstruction complète de l'état interne du Mersenne Twister à partir de 624 observations. Ces attaques illustrent un principe fondamental : de bonnes propriétés statistiques ne garantissent pas la sécurité cryptographique.

Mots-clés : PRNG, CSPRNG, LCG, Mersenne Twister, entropie de Shannon, tests statistiques, cryptanalyse, sécurité, graine, inversion de tempering.

Note de responsabilité

Les attaques présentées dans ce document sont **strictement pédagogiques**. Elles visent à illustrer les faiblesses théoriques de certains algorithmes dans un cadre académique contrôlé. Toute reproduction de ces techniques sur des systèmes réels sans autorisation explicite est **illégale** et passible de poursuites judiciaires au titre de la loi Godfrain (art. 323-1 et suivants du Code pénal). Les auteurs déclinent toute responsabilité quant à une utilisation malveillante du contenu de ce rapport.

Sommaire

Table des matières

Informations administratives **1**

Résumé **2**

Sommaire **3**

Introduction 4
1. Générateurs 5
1.1 Linear Congruential Generator (LCG) 5
1.2 Mersenne Twister (MT19937) 6
1.3 Transformée de Box-Muller 7
1.4 Blum-Blum-Shub (BBS) 8
1.5 Hash_DRBG (NIST SP 800-90A) 8
1.6 os.urandom 9
1.7 XOR NRBG (Non-Deterministic Random Bit Generator) 10
2. Tests statistiques 11
2.1 Entropie de Shannon 11
2.2 Test du Chi-carré 12
2.3 Autocorrélation 12
2.4 Test de Kolmogorov-Smirnov (KS) 13
3. Attaques 15
3.1 Récupération de la graine LCG 15
3.2 Reconstruction d'état du Mersenne Twister (MT19937) 16
4. Recommandations et analyse critique 18
4.1 Les tests statistiques ne suffisent pas 18
4.2 Principe de Kerckhoffs 18
4.3 Cas réel : les casinos en ligne et le Mersenne Twister 18
4.4 Tableau comparatif des générateurs 19
4.5 Règles fondamentales 19
5. Conclusion 20
Bibliographie 21

Introduction

Ce document présente l'étude de sept générateurs de nombres pseudo-aléatoires (PRNG), quatre tests statistiques permettant d'évaluer leur qualité, et deux attaques démontrant les faiblesses cryptographiques de certains d'entre eux. L'objectif est de comprendre les différences fondamentales entre les générateurs à usage général et les générateurs cryptographiquement sûrs (CSPRNG), et de montrer pourquoi l'utilisation d'un PRNG non cryptographique dans un contexte de sécurité constitue une vulnérabilité grave.

Les générateurs étudiés sont regroupés en quatre catégories :

- **PRNG non cryptographiques** : LCG, Mersenne Twister
- **Distribution gaussienne** : transformée de Box-Muller
- **CSPRNG** : Blum-Blum-Shub, Hash_DRBG, os.urandom
- **Hybride non-déterministe** : XOR NRBG

1. Générateurs

1.1 Linear Congruential Generator (LCG)

Principe Le LCG est l'un des plus anciens et des plus simples PRNG. Il repose sur une relation de récurrence linéaire :

```
X_{n+1} = (a * X_n + c) mod m
```

où :

- `X_0` est la **graine** (seed), valeur initiale secrète
- `a` est le **multiplicateur**
- `c` est l'**incrément**
- `m` est le **module** (définit la période maximale)

La suite est entièrement déterministe : connaissant `X_0`, `a`, `c` et `m`, on peut reproduire toute la séquence.

Implémentation

```python
def lcg(seed, a, c, m, n):
    x = seed
    results = []
    for _ in range(n):
        x = (a * x + c) % m
        results.append(x)
    return results
```

La fonction prend une graine, les paramètres `(a, c, m)` et le nombre de valeurs `n` à générer. Elle retourne une liste d'entiers dans `[0, m-1]`.

Paramètres standards

Nom	a	c	m	Source
glibc	1 103 515 245	12 345	2^31	Bibliothèque standard C
RANDU	65 539	0	2^31	IBM (1960s, mauvais)
MMIX (Knuth)	6 364 136 223 846 793 005	1 442 695 040 888 963 407	2^64	The Art of Computer Programming

Propriétés

- **Période maximale** : `m`
- **Avantage** : extrêmement rapide
- **Faiblesse principale** : les valeurs successives sont corrélées linéairement.
- **Usage** : simulation, jeux vidéo — **jamais** en cryptographie

1.2 Mersenne Twister (MT19937)

Principe Le Mersenne Twister est le PRNG le plus utilisé dans les langages de programmation (`random` en Python, `rand()` en C++). Sa période est le nombre premier de Mersenne `2^19937 - 1`.

L'algorithme se décompose en trois phases :

1. **Initialisation** : à partir d'une graine de 32 bits, construction d'un état interne de 624 mots de 32 bits
2. **Twist** : transformation mélangeant les 624 valeurs entre elles (une fois toutes les 624 extractions)
3. **Tempering** : transformation de sortie améliorant la distribution statistique

Implémentation **Constantes MT19937** :

```
W=32, N=624, M=397, R=31
A=0x9908B0DF, F=1812433253
U=11, D=0xFFFFFFFF, S=7,  B=0x9D2C5680
T=15, C=0xEFC60000, L=18
```

Initialisation — construction de l'état à partir de la graine :

```python
def init(seed):
    state = [seed & 0xFFFFFFFF]
    for i in range(1, N):
        prev = state[i - 1]
        state.append((F * (prev ^ (prev >> 30)) + i) & 0xFFFFFFFF)
    return state
```

Twist — mélange de l'état interne :

```python
def twist(state):
    for i in range(N):
        bit_haut = state[i] & 0x80000000
        bits_bas = state[(i + 1) % N] & 0x7FFFFFFF
        x = bit_haut | bits_bas
        x_shifted = x >> 1
        if x & 1:
            x_shifted ^= A
        state[i] = state[(i + M) % N] ^ x_shifted
    return state
```

Pour chaque position `i`, le bit de poids fort de `state[i]` est combiné avec les 31 bits de poids faible de `state[i+1]`, puis XORé avec `state[i+397]`. Si le bit de poids faible est 1, un XOR supplémentaire avec `A` est appliqué.

Tempering — amélioration de la distribution en sortie :

```python
def temper(y):
    y ^= (y >> U) & D
    y ^= (y << S) & B
    y ^= (y << T) & C
    y ^= y >> L
    return y & 0xFFFFFFFF
```

Génération :

```python
def generate(seed, n):
```

```
    state = init(seed)
    index = N
    results = []
    for _ in range(n):
        if index >= N:
            twist(state)
            index = 0
        results.append(temper(state[index]))
        index += 1
    return results
```

L'état est parcouru séquentiellement. Toutes les 624 extractions, un twist régénère l'état complet.

Propriétés

- **Période** : `2^19937 - 1`
- **Avantage** : passe la majorité des tests statistiques standards
- **Faiblesse principale** : l'état interne est entièrement reconstructible à partir de 624 sorties consécutives (cf. section 3.2)

1.3 Transformée de Box-Muller

Principe La transformée de Box-Muller convertit deux valeurs uniformes indépendantes `U1, U2` dans `(0, 1)` en deux valeurs gaussiennes indépendantes suivant `N(0, 1)`. Ce n'est pas un générateur autonome : il nécessite un PRNG.

Formules

```
Z0 = sqrt(-2 * ln(U1)) * cos(2pi * U2)
Z1 = sqrt(-2 * ln(U1)) * sin(2pi * U2)
```

Implémentation

```
def box_muller(u1, u2):
    if u1 <= 0:
        u1 = 1e-10
    if u2 <= 0:
        u2 = 1e-10
    r = math.sqrt(-2.0 * math.log(u1))
    theta = 2.0 * math.pi * u2
    z0 = r * math.cos(theta)
    z1 = r * math.sin(theta)
    return z0, z1
```

La garde `u1 > 0` évite `ln(0)`. La fonction `box_muller_series` génère `n` valeurs gaussiennes en chaînant les paires `(U1, U2)` issues d'un générateur uniforme :

```
def box_muller_series(uniform_rng, seed, n):
    n_uniform = ((n + 1) // 2) * 2
    uniforms = uniform_rng(seed, n_uniform)
    results = []
    for i in range(0, len(uniforms) - 1, 2):
        z0, z1 = box_muller(uniforms[i], uniforms[i + 1])
```

```
        results.extend([z0, z1])
    return results[:n]
```

Propriétés

- **Distribution** : `N(0, 1)`, utilisable en simulation de phénomènes naturels
- **Dépendance** : la qualité de la sortie dépend entièrement du générateur uniforme sous-jacent
- **Non cryptographique** : faiblesses du générateur en amont

---

1.4 Blum-Blum-Shub (BBS)

Principe BBS est un générateur cryptographiquement sûr reposant sur la difficulté de la factorisation d'entiers. Il utilise le module `M = p x q`, où `p` et `q` sont deux grands nombres premiers de Blum satisfaisant `p = q = 3 (mod 4)`.

Formule

```
x_{n+1} = (x_n)^2 mod M
```

Seul le **bit de poids faible** de `x_n` est extrait à chaque itération.

Implémentation

```
def bbs(seed, p, q, n):
    M = p * q
    x = (seed * seed) % M
    results = []
    for _ in range(n):
        x = (x * x) % M
        results.append(x & 1)
    return results
```

Les paramètres de test utilisés (`p = 499`, `q = 547`) sont des petits premiers de Blum à but pédagogique. En pratique, `p` et `q` doivent être des nombres de plusieurs centaines de bits (1024 bits).

Propriétés

- **Sécurité** : sous l'hypothèse de difficulté de la factorisation
- **Lenteur** : chaque bit nécessite une exponentiation modulaire
- **Usage** : protocoles cryptographiques nécessitant une preuve de sécurité formelle

---

1.5 Hash_DRBG (NIST SP 800-90A)

Principe Le Hash_DRBG (Hash-based Deterministic Random Bit Generator) est un CSPRNG standardisé par le NIST (SP 800-90A). Il est basé sur SHA-256 et maintient un état interne composé de deux vecteurs `V` et `C` de 55 octets (440 bits = `seedlen` pour SHA-256).

L'architecture comprend trois opérations :

1. **Instanciation** (`drbg_instantiate`) : initialisation de l'état à partir d'une entropie et d'un nonce
2. **Génération** (`drbg_generate`) : production des octets pseudo-aléatoires en itérant SHA-256 sur `V`
3. **Reseed** (`drbg_reseed`) : mise à jour de l'état avec de l'entropie fraîche

Implémentation **Fonction de dérivation** (`Hash_df`) :

```
def _hash_df(input_data, num_bytes):
    hash_len = 32
    num_blocks = (num_bytes + hash_len - 1) // hash_len
    result = b""
    for counter in range(1, num_blocks + 1):
        to_hash = (counter.to_bytes(1, "big")
                   + num_bytes.to_bytes(4, "big") + input_data)
        result += _sha256(to_hash)
    return result[:num_bytes]
```

Génération :

```
def drbg_generate(state, num_bytes):
    m = (num_bytes + 32 - 1) // 32
    W = b""
    data = state["V"]
    for _ in range(m):
        W += _sha256(data)
        int_data = (int.from_bytes(data, "big") + 1) % (2 ** (len(data) * 8))
        data = int_data.to_bytes(len(state["V"]), "big")
    output = W[:num_bytes]
    # Mise a jour de l'etat...
    return output, state
```

La fonction raccourci utilisée dans le projet :

```
def drbg_generate_bytes(n, entropy=None, nonce=None):
    state = drbg_instantiate(entropy=entropy, nonce=nonce)
    output, _ = drbg_generate(state, n)
    return output
```

Propriétés

- **Standardisé** : conforme NIST SP 800-90A, certifié FIPS
- **Résistance à la prédiction** : connaître les sorties passées ne permet pas de prédire les suivantes
- **Reseed** : mécanisme de rafraîchissement de l'entropie
- **Usage** : génération de clés, nonces, IV en cryptographie

---

1.6 os.urandom

Principe `os.urandom` est l'interface Python vers le générateur aléatoire du système d'exploitation. Sous Linux, il interroge `/dev/urandom`, lequel collecte de l'entropie à partir de sources matérielles (interruptions clavier, mouvements souris, timings disque, etc.) et logicielles (RDRAND sur processeurs Intel/AMD).

Implémentation

```python
def os_generate_bytes(n):
    return os.urandom(n)
```

Le noyau maintient un pool d'entropie et fournit des octets cryptographiquement sûrs via cet appel système.

Propriétés

- **Entropie réelle** : basée sur des sources physiques non prédictibles
- **Cryptographiquement sûr** : recommandé pour tout usage cryptographique en Python
- **Simplicité** : aucune gestion d'état côté application
- **Usage** : génération de clés, tokens, mots de passe

1.7 XOR NRBG (Non-Deterministic Random Bit Generator)

Principe Le XOR NRBG combine plusieurs générateurs via l'opération XOR. La propriété fondamentale est la suivante : si **au moins un** des générateurs sources produit une sortie véritablement aléatoire et indépendante, alors la sortie combinée reste aléatoire, quelles que soient les propriétés des autres sources.

```
sortie[i] = gen1[i] XOR gen2[i] XOR ... XOR genK[i]
```

Implémentation

```python
def xor_nrbg(generators, seeds, n):
    if len(generators) != len(seeds):
        raise ValueError("Nombre generateurs != nombre graines")
    outputs = [gen(seed, n) for gen, seed in zip(generators, seeds)]
    result = []
    for i in range(n):
        xor_val = 0
        for output in outputs:
            xor_val ^= output[i]
        result.append(xor_val)
    return result
```

Des variantes opèrent sur des bits (`xor_combine_bits`) ou sur des octets (`xor_combine_bytes`).

Propriétés

- **Robustesse** : tolérant à la défaillance d'un ou plusieurs générateurs
- **Flexibilité** : combinaison de sources hétérogènes (PRNG, CSPRNG, capteurs)
- **Limitation** : si *toutes* les sources sont biaisées ou compromises, la sortie l'est aussi

2. Tests statistiques

Les tests sont implémentés dans `STATISTICS/test_statistique.py` et évaluent la qualité d'une séquence d'octets `[0, 255]`. Chaque test produit un verdict PASS/FAIL selon un seuil calibré à `alpha = 0.05`.

2.1 Entropie de Shannon

Principe L'entropie de Shannon mesure la quantité moyenne d'information contenue dans chaque symbole de la séquence. Pour une source produisant des octets avec la distribution de probabilité `{p_i}` :

```
H = - sum p_i * log2(p_i)   (en bits/octet)
```

Pour une distribution **parfaitement uniforme** sur 256 valeurs, `p_i = 1/256` et `H = log2(256) = 8 bits/octet` (maximum théorique). Toute déviation par rapport à l'uniformité réduit `H`.

Implémentation

```python
def shannon_entropy(data):
    freq = Counter(data)
    n = len(data)
    entropy = 0.0
    for count in freq.values():
        p = count / n
        if p > 0:
            entropy -= p * math.log2(p)
    return entropy
```

Le seuil de passage est `H > 7.9 bits/octet` (soit 98.75 % du maximum théorique).

Application aux générateurs

Générateur	H (bits/octet)	Verdict
os.urandom	~7.97	PASS
Hash_DRBG	~7.97	PASS
MT19937	~7.95	PASS
LCG (glibc)	~7.94	PASS
BBS	~7.90	PASS
Box-Muller	variable	dépend du générateur amont

Le LCG et le MT19937 obtiennent une entropie satisfaisante : l'entropie de Shannon ne suffit pas à détecter leurs faiblesses structurelles.

2.2 Test du Chi-carré

Principe Le test du chi-carré évalue l'**uniformité de la distribution** des octets. Sous l'hypothèse nulle `H_0` (distribution uniforme), chacune des 256 valeurs possibles doit apparaître avec une fréquence attendue `E = n / 256`.

La statistique du test est :

```
X2 = sum_{i=0}^{255} (O_i - E)^2 / E
```

où `O_i` est le nombre d'occurrences observées de la valeur `i`. Sous `H_0`, `X2` suit une loi du chi-carré à `df = 255` degrés de liberté. Pour `alpha = 0.05`, la valeur critique est `X2_critique = 293.25`.

Implémentation

```python
def chi_squared_test(data, p=0.05):
    freq = Counter(data)
    n = len(data)
    expected = n / 256
    chi2 = 0.0
    for i in range(256):
        observed = freq.get(i, 0)
        chi2 += ((observed - expected) ** 2) / expected
    critical_value = 293.25  # pour df=255, alpha=0.05
    return {
        'chi2': chi2,
        'degrees_freedom': 255,
        'critical_value': critical_value,
        'status': 'PASS' if chi2 < critical_value else 'FAIL'
    }
```

Le test passe si `X2 < 293.25`.

Application aux générateurs Un générateur uniforme de qualité doit avoir `X2` proche de 255 (la valeur attendue). Les CSPRNG (os.urandom, Hash_DRBG) passent systématiquement. Le LCG peut échouer selon les paramètres utilisés.

2.3 Autocorrélation

Principe L'autocorrélation mesure la dépendance linéaire entre une séquence et une version décalée d'elle-même. Pour un générateur idéal, les valeurs successives doivent être **indépendantes**, donc le coefficient d'autocorrélation doit être proche de 0 pour tout décalage (lag).

Le coefficient pour un décalage `k` est :

```
r(k) = Cov(X_i, X_{i+k}) / Var(X)
     = [ sum (X_i - mu)(X_{i+k} - mu) ] / [ sum (X_i - mu)^2 ]
```

où `mu` est la moyenne empirique.

Implémentation

```python
def autocorrelation(data, lag=1):
    n = len(data) - lag
    moy = sum(data) / len(data)
    variance = sum((x - moy)**2 for x in data)
    if variance == 0:
        return 0.0
    covariance = sum((data[i] - moy) * (data[i + lag] - moy) for i in range(n))
    r = covariance / variance
    return r
```

La fonction `autocorrelation_test` teste plusieurs décalages (`lags = [1, 8, 16, 32]`) et retourne PASS si `|r(k)| < 0.05` pour chacun.

Application aux générateurs Le LCG présente des autocorrélations significatives pour les petits lags, confirmant la corrélation linéaire entre valeurs consécutives. Le MT19937 présente des corrélations détectables sur les octets de poids faible. Les CSPRNG (Hash_DRBG, os.urandom) restent dans les limites acceptables.

2.4 Test de Kolmogorov-Smirnov (KS)

Principe Le test de Kolmogorov-Smirnov compare la **fonction de répartition empirique** `F_n(x)` des données à la fonction de répartition théorique `F(x)` de la distribution uniforme sur `[0, 255]`.

La statistique du test est la distance maximale entre les deux fonctions :

`D_n = max |F_n(x) - F(x)|`

où `F_n(x) = (nombre de valeurs <= x) / n` et `F(x) = x / 255`.

Hypothèses

- **H0** : les données suivent la distribution uniforme
- **H1** : les données ne suivent pas la distribution uniforme

On rejette `H0` si `D_n > D_critique`, avec `D_critique = 1.36 / sqrt(n)` pour `alpha = 0.05`.

Implémentation

```python
def kolmogorov_smirnov_test(data):
    normalized = sorted([x / 255.0 for x in data])
    n = len(normalized)
    max_diff = 0.0
    for i, value in enumerate(normalized):
        empi = (i + 1) / n    # F_n(x)
        theo = value           # F(x) = x/255 normalise dans [0,1]
        diff = abs(empi - theo)
        max_diff = max(max_diff, diff)
    critical_value = 1.36 / math.sqrt(n)
    return {
        'D': max_diff,
        'critical_value': critical_value,
        'n': n,
```

```
        'status': 'PASS' if max_diff < critical_value else 'FAIL'
    }
```

Application aux générateurs Le test KS est non-paramétrique : il ne suppose aucune forme particulière pour la distribution et est sensible aussi bien aux écarts de position que de forme. Les données constantes ou avec peu de valeurs distinctes produisent un `D` élevé et échouent immédiatement.

3. Attaques

3.1 Récupération de la graine LCG

Modèle de menace L'attaquant observe quelques sorties consécutives du LCG et connaît (ou a deviné) les paramètres publics `(a, c, m)`. Son objectif est de retrouver la graine `X_0` pour prédire toutes les sorties futures.

Trois méthodes sont implémentées dans `ATTACKS/lcg_seed_recovery.py`.

Méthode 1 : Résolution algébrique C'est la méthode la plus efficace. Sachant que `X_1 = (a * X_0 + c) mod m`, on inverse :

```
X_0 = (X_1 - c) * a^{-1} mod m
```

où `a^{-1}` est l'inverse modulaire de `a` modulo `m`, calculé via `pow(a, -1, m)` (algorithme d'Euclide étendu, complexité `O(log m)`).

```python
def recover_seed_algebrique(x1, x2, x3, a, c, m):
    a_inv = pow(a, -1, m)
    x0 = ((x1 - c) * a_inv) % m
    return x0
```

Résultat : la graine est récupérée en une seule opération à partir de la première sortie observée. La prédiction des sorties futures est parfaite à 100 %.

Méthode 2 : Force brute Si l'espace des graines est restreint (timestamp, PID, compteur), on teste exhaustivement toutes les candidates :

```python
def recover_seed_bruteforce(outputs, a, c, m, seed_max=1000000):
    for candidate in range(seed_max):
        generated = lcg(candidate, a, c, m, len(outputs))
        if generated == outputs:
            return candidate
    return None
```

Complexité : `O(seed_max x len(outputs))`. Pour un espace de 100 000 graines et 5 sorties de validation, l'attaque s'exécute en moins d'une seconde.

Méthode 3 : Known-plaintext (chiffrement XOR) Si le LCG est utilisé comme générateur de flux pour un chiffrement XOR (`C = P XOR LCG_keystream`) et que l'attaquant connaît un couple `(P, C)`, il récupère directement le keystream :

```
K = P XOR C
```

puis recherche la graine par force brute sur les octets du keystream :

```python
def recover_seed_from_xor(plaintext, ciphertext, a, c, m, seed_max=100000):
    keystream = xor_bytes(plaintext, ciphertext)
    keystream_ints = list(keystream)
    for candidate in range(seed_max):
        generated = lcg(candidate, a, c, m, len(keystream))
        generated_bytes = [x % 256 for x in generated]
        if generated_bytes == keystream_ints:
            return candidate
    return None
```

Conclusion sur le LCG Le LCG est **totalement compromis** dès qu'une seule sortie est observée (méthode algébrique). Utiliser un LCG pour générer des clés, nonces ou IV constitue une faille de sécurité critique.

3.2 Reconstruction d'état du Mersenne Twister (MT19937)

Modèle de menace L'attaquant observe **624 sorties consécutives** de 32 bits du MT19937. Son objectif est de reconstruire l'état interne complet (`N = 624` mots de 32 bits) pour prédire toutes les sorties futures, indéfiniment.

Principe de l'attaque Chaque sortie du MT est `sortie = temper(state[i])`. Le tempering est une suite de quatre opérations XOR avec décalages et masques, et chacune est **inversible bit à bit**.

Inversion du tempering Pour inverser `y ^= (y >> shift)`, on reconstruit les bits de gauche à droite — les `shift` bits de poids fort ne sont pas modifiés, chaque bit suivant se retrouve par XOR avec le bit déjà reconstruit `shift` positions plus haut :

```python
def invert_right_shift_xor(y, shift):
    result = 0
    for i in range(32):
        bit_pos = 31 - i
        if i < shift:
            bit = (y >> bit_pos) & 1
        else:
            prev_bit = (result >> (bit_pos + shift)) & 1
            current_bit = (y >> bit_pos) & 1
            bit = current_bit ^ prev_bit
        result |= (bit << bit_pos)
    return result
```

Pour inverser `y ^= (y << shift) & mask`, même principe de droite à gauche, en tenant compte du masque :

```python
def invert_left_shift_xor_mask(y, shift, mask):
    result = 0
    for i in range(32):
        if i < shift:
            bit = (y >> i) & 1
        else:
            prev_bit = (result >> (i - shift)) & 1
            mask_bit = (mask >> i) & 1
            current_bit = (y >> i) & 1
            bit = current_bit ^ (prev_bit & mask_bit)
        result |= (bit << i)
    return result
```

La fonction `untemper` inverse les quatre étapes **dans l'ordre inverse** :

```python
def untemper(y):
    y = invert_right_shift_xor(y, L)          # inverse y ^= y >> L
    y = invert_left_shift_xor_mask(y, T, C)   # inverse y ^= (y << T) & C
    y = invert_left_shift_xor_mask(y, S, B)   # inverse y ^= (y << S) & B
```

```
    y = invert_right_shift_xor(y, U)          # inverse y ^= (y >> U) & D
    return y
```

Reconstruction de l'état On applique `untemper` sur chacune des 624 sorties observées :

```
def recover_state(outputs):
    if len(outputs) < N:
        raise ValueError(f"Besoin de {N} sorties, recu {len(outputs)}")
    state = []
    for i in range(N):
        state.append(untemper(outputs[i]))
    return state
```

Prédiction des sorties futures L'état reconstruit correspond à l'état interne **avant le prochain twist**. La prédiction s'effectue en appliquant `twist` puis `temper` :

```
def predict_next(state, index, n):
    from GENERATORS.PRNG_non_cryptographics.mersenne_twister import twist, temper
    state = state.copy()
    predictions = []
    for _ in range(n):
        if index >= N:
            state = twist(state)
            index = 0
        y = temper(state[index])
        predictions.append(y)
        index += 1
    return predictions
```

Pour prédire les sorties suivant immédiatement les 624 observées, il faut appeler `predict_next(state, N, n)` afin de forcer le twist dès le premier appel.

Résultats L'attaque produit une **prédiction parfaite à 100 %** des sorties futures.

Sorties observées	Attaque possible
< 624	Non — état incomplet
$>= 624$	Oui — reconstruction complète

Complexité : `O(624)` inversions de tempering, exécutable en moins d'une milliseconde.

Conclusion sur le MT19937 Malgré ses excellentes propriétés statistiques (période `2^19937 - 1`), le MT19937 est **totalement prévisible** après 624 observations. Il ne doit **jamais** être utilisé en cryptographie.

4. Recommandations et analyse critique

4.1 Les tests statistiques ne suffisent pas

Une idée reçue courante est qu'un générateur qui "passe les tests statistiques" est sûr pour un usage cryptographique. **C'est faux.** Le LCG et le MT19937 passent la quasi-totalité des tests statistiques classiques (Shannon, chi-carré, Diehard, TestU01) tout en étant trivialement cassables :

- Le LCG est compromis en **une seule opération algébrique** à partir d'une sortie observée.
- Le MT19937 est entièrement reconstruit en **moins d'une milliseconde** après 624 observations.

Les tests statistiques mesurent l'*apparence* aléatoire d'une séquence. Ils ne mesurent pas la résistance aux attaques cryptographiques, qui requiert des propriétés d'imprévisibilité bien plus exigeantes.

4.2 Principe de Kerckhoffs

Le principe de Kerckhoffs (1883) stipule que la **sécurité d'un système ne doit reposer que sur le secret de la clé** (ou ici, de la graine), jamais sur le secret de l'algorithme. En d'autres termes :

> *"Un système doit être sûr même si tout, sauf la clé, est connu de l'ennemi."*

Un bon générateur cryptographique doit donc rester imprévisible même lorsque :

1. L'algorithme complet est connu publiquement
2. L'attaquant peut observer un grand nombre de sorties passées
3. L'attaquant dispose d'une puissance de calcul importante

Le LCG et le MT19937 violent ce principe : connaître l'algorithme et quelques sorties suffit à tout prédire. À l'inverse, `os.urandom` et Hash_DRBG sont conçus pour résister à ces conditions.

4.3 Cas réel : les casinos en ligne et le Mersenne Twister

L'exploitation du Mersenne Twister ne relève pas uniquement de la théorie. En **2007-2010**, plusieurs incidents majeurs ont mis en évidence le risque concret de son usage dans des contextes sensibles.

Des chercheurs en sécurité ont découvert que plusieurs sites de poker en ligne utilisaient MT19937, souvent initialisé avec le timestamp Unix (précision à la seconde), pour mélanger les cartes et distribuer les mains. L'espace des graines possibles se réduisait alors à quelques dizaines de milliers de valeurs — un espace attaquable par force brute en quelques secondes.

Déroulement de l'attaque :

1. L'attaquant observe les premières cartes visibles à la table (cartes communautaires au Texas Hold'em)
2. Ces cartes lui permettent de contraindre l'espace des graines possibles du MT
3. En testant les graines candidates, il retrouve celle qui produit exactement les cartes observées
4. Il prédit alors l'intégralité du sabot restant — y compris les cartes privées de ses adversaires

Le résultat : l'attaquant jouait avec une connaissance parfaite de toutes les mains, équivalant à une tricherie informatique totale. Des groupes organisés ont exploité cette faille pour gagner des sommes considérables avant d'être détectés.

Cette affaire illustre qu'une erreur de choix de générateur peut avoir des conséquences financières et légales directes, même lorsque l'algorithme est "statistiquement bon".

4.4 Tableau comparatif des générateurs

Générateur	Vitesse	Sécurité cryptographique	Tests statistiques	Usage recommandé
LCG	Très rapide	**Nulle** (1 sortie)	Passable	Simulation uniquement
MT19937	Rapide	**Nulle** (624 sorties)	Excellents	Simulation uniquement
Box-Muller	Rapide (amont)	Héritée du PRNG amont	N/A (gaussienne)	Simulation (variables normales)
BBS	Très lent	Élevée (théorique)	Bonne (grands params.)	Protocoles formels
Hash_DRBG	Moyen	**Très élevée** (NIST)	Excellents	Cryptographie, certifié FIPS
os.urandom	Moyen	**Très élevée** (entropie OS)	Excellents	Cryptographie, référence
XOR NRBG	Variable	Dépend des sources	Bonne	Haute disponibilité

4.5 Règles fondamentales

1. **Ne jamais utiliser LCG ou MT19937** pour générer des clés, nonces, IV ou tokens d'authentification
2. **La sécurité doit reposer sur la graine**, pas sur la confidentialité de l'algorithme (principe de Kerckhoffs)
3. **Un bon CSPRNG doit rester imprévisible** même si l'algorithme est public et si l'attaquant observe les sorties
4. Préférer `os.urandom` ou le module `secrets` de Python pour tout besoin de sécurité
5. Utiliser Hash_DRBG (NIST SP 800-90A) pour les applications nécessitant une certification formelle
6. Considérer le reseed régulier pour les CSPRNG à longue durée de vie

5. Conclusion

Ce projet a permis d'étudier en profondeur le spectre des générateurs de nombres pseudo-aléatoires, depuis les algorithmes historiques jusqu'aux standards cryptographiques modernes. Les résultats obtenus confirment une distinction fondamentale qui ne doit pas être ignorée en pratique.

Les générateurs non cryptographiques — LCG et Mersenne Twister — présentent d'excellentes propriétés statistiques. Ils passent l'ensemble des tests mis en œuvre (Shannon, chi-carré, autocorrélation, Kolmogorov-Smirnov) avec des scores comparables aux CSPRNG. Pourtant, les deux attaques implémentées démontrent qu'ils sont **totalement prévisibles** dès lors qu'un attaquant observe quelques sorties : une opération algébrique suffit pour le LCG, 624 observations pour le MT19937. Ces générateurs violent le principe de Kerckhoffs et ne doivent jamais être utilisés dans un contexte de sécurité.

À l'opposé, les générateurs cryptographiquement sûrs — Hash_DRBG et os.urandom — satisfont à la fois les critères statistiques et les exigences de sécurité. Leur conception garantit l'imprévisibilité même en connaissance totale de l'algorithme.

L'incident des casinos en ligne exploitant le Mersenne Twister illustre concrètement le coût d'un choix inapproprié de générateur : des pertes financières réelles, des poursuites judiciaires et une atteinte à la réputation. Ce n'est pas une vulnérabilité théorique — c'est une faille exploitée avec succès.

La leçon centrale de ce travail peut se résumer ainsi : **les tests statistiques évaluent l'apparence aléatoire, mais ne mesurent pas la sécurité cryptographique**. Ces deux propriétés sont indépendantes, et leur confusion est la source de nombreuses vulnérabilités en production.

Bibliographie

Knuth, D. E. (1997). *The Art of Computer Programming, Volume 2 : Seminumerical Algorithms*. Addison-Wesley.

Matsumoto, M., & Nishimura, T. (1998). Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, *8*(1), 3–30.

Marsaglia, G. (1968). Random numbers fall mainly in the planes. *Proceedings of the National Academy of Sciences*, *61*(1), 25–28.

National Institute of Standards and Technology. (2015). *NIST SP 800-90A Rev. 1 : Recommendation for random number generation using deterministic random bit generators*. U.S. Department of Commerce.

Kolmogorov, A. N. (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, *4*, 83–91.